# Datos compartidos

## condiciones de carrera y synchronized

**Pregunta que dejó la presentación anterior:**

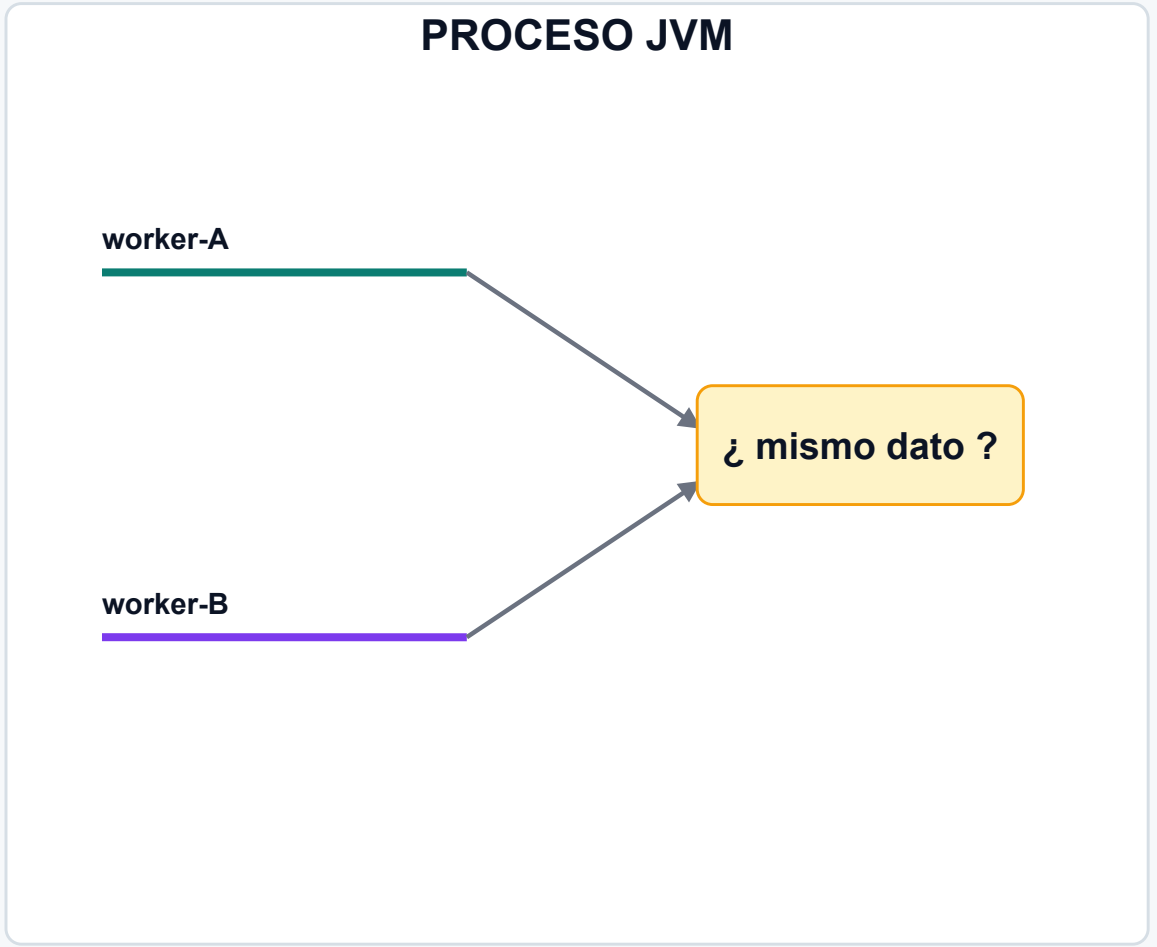
**Dos hilos pueden estar perfectamente escritos por separado.**
**Si ambos modifican el mismo dato,**
**¿basta con eso para que el resultado conjunto sea correcto?**

### En esta presentación aprenderemos a:

- identificar estado mutable compartido
- explicar una actualización perdida
- reconocer una condición de carrera
- localizar una sección crítica
- protegerla con synchronized
- comprobar que el resultado conjunto es correcto

**Resultado final**

**Thread A + Thread B + mismo estado**
**↓**
**resultado correcto**




**crear hilos ✓ → ejecutarlos ✓ → ¿compartir datos correctamente?**

# De un dato local a un objeto compartido

## En la presentació anterior:

```
override fun run() {
    var segundos = 0
    // ...
}
```

**segundos es local a esa ejecución.**

**Thread A**

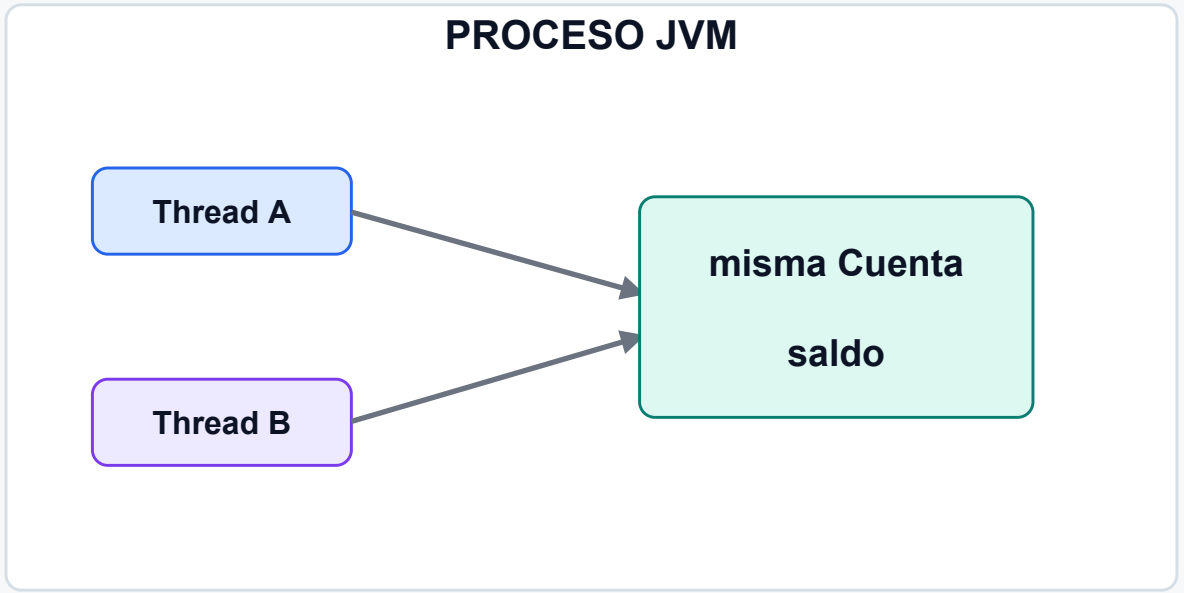
**segundosA**

**Thread B**

**segundosB**

## Ahora

```
val cuenta = Cuenta()
```




## Estado compartido

**Varios hilos pueden acceder al mismo objeto y a su estado cuando comparten esa referencia.**

**Compartir un dato ≠ compartirlo correctamente**

# Dinero en el banco: ¿qué resultado debe salir?

```
class Cuenta {
    var saldo = 0

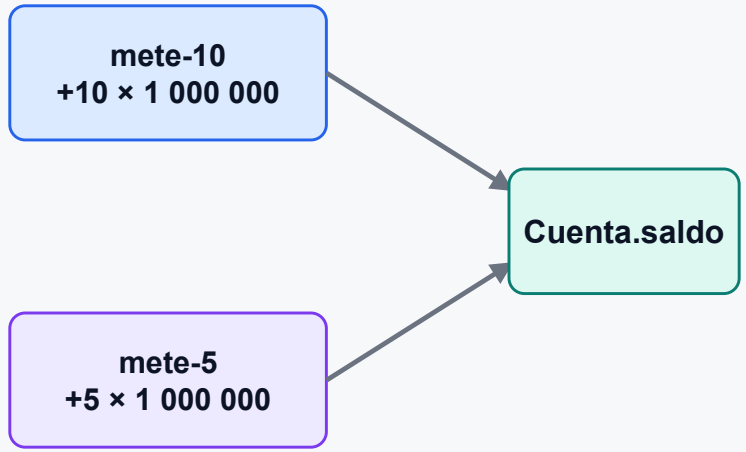
    fun ingresar(cantidad: Int) {
        saldo += cantidad
    }
}
```

**Los dos utilizan la misma cuenta.**

```
val n = 1_000_000
val cuenta = Cuenta()

val mete10 = Thread {
    repeat(n) {
        cuenta.ingresar(10)
    }
}
mete10.name = "mete-10"

val mete5 = Thread {
    repeat(n) {
        cuenta.ingresar(5)
    }
}
mete5.name = "mete-5"
```




## Invariante final

**saldo esperado**
**= n·10 + n·5**
**= 1 000 000·15**
**= 15 000 000**

**Sea cual sea el orden de ejecución, si no se pierde ningún ingreso, al terminar debe haber 15 000 000.**

# Probemos primero el programa sin protección (1/2)

## Programa completo que ejecutaremos

```
fun main() {
    val n = 1_000_000
    val cuenta = Cuenta()

    val mete10 = Thread {
        repeat(n) { cuenta.ingresar(10) }
    }
    mete10.name = "mete-10"

    val mete5 = Thread {
        repeat(n) { cuenta.ingresar(5) }
    }
    mete5.name = "mete-5"

    mete10.start()
    mete5.start()

    mete10.join()
    mete5.join()

    println("Esperado: 15_000_000")
    println("Observado: ${cuenta.saldo}")
}
```

**Fijémonos en tres decisiones:**

**1. Una sola cuenta compartida**

**2. Ambos workers se arrancan antes de esperar**

**3. main comprueba después de los dos join()**

**No estamos protegiendo todavía el saldo.**

# Probemos primero el programa sin protección (2/2)

start mete-10
start mete-5

---

join mete-10
join mete-5

---

comprobar saldo

## Ejecutamos varias veces

| Ejecución | Esperado | Observado |
|---|---|---|
| 1 | 15 000 000 | ? |
| 2 | 15 000 000 | ? |
| 3 | 15 000 000 | ? |

**Si ahora sale 15 000 000, no hemos demostrado ausencia de carrera.**
**Necesitamos repetir y razonar sobre un interleaving posible.**

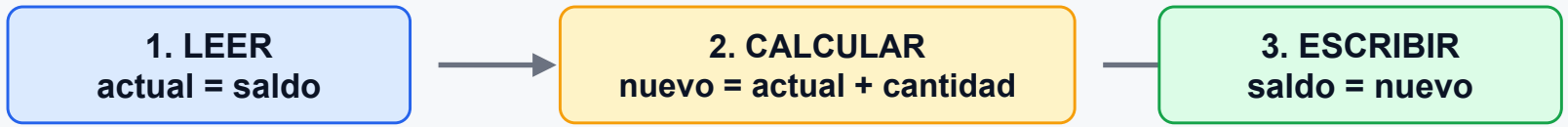
**Una ejecución correcta no demuestra que el programa sea correcto.**

**start A y start B antes de cualquier join**
**→ estudiamos concurrencia real entre los workers.**

# Una línea de Kotlin no implica un único paso

```
saldo += cantidad
```

**Para razonar sobre la concurrencia podemos modelarlo como:**




**Igualmente:**

```
contador++
```

leer → sumar 1 → escribir

## Atomicidad

Una operación es atómica respecto de otros hilos cuando se comporta como una unidad indivisible para la coordinación que estamos analizando.

```
saldo += cantidad
```

**no debe asumirse atómica**

# Actualización perdida: reconstruyamos el fallo

## Saldo inicial = 0

| Thread A · +10 | saldo | Thread B · +5 |
|---|---|---|
| lee 0 | 0 | |
| | 0 | lee 0 |
| calcula 10 | 0 | |
| | 0 | calcula 5 |
| escribe 10 | 10 | |
| | **5** | escribe 5 |

**esperado = 15**
**obtenido = 5**

**La actualización de A ha quedado sobrescrita.**

**Ningún cálculo individual estaba mal.**
**El fallo aparece por el interleaving sobre el mismo estado.**

**No requiere paralelismo físico:**
**basta con que los pasos de A y B se intercalen.**

# Ahora podemos darle nombre: condición de carrera

## Tenemos condición de carrera cuando:

La corrección del resultado depende de un orden o interleaving no controlado de operaciones concurrentes que acceden a estado compartido.

concurrencia
+
estado mutable compartido
+
accesos que interfieren
sin coordinación adecuada

## No confundir

**Variabilidad permitida**

A imprime primero
o
B imprime primero

Ambos resultados pueden ser correctos.

**Carrera problemática**

algún interleaving
↓
viola la invariante

**Orden variable ≠ error automáticamente**

# Dos propiedades que debemos tener presentes

**Atomicidad**

**¿Puede otro hilo intercalarse dentro de una operación que necesitamos tratar como una unidad?Ja**

**LEER → CALCULAR → ESCRIBIR**

**no debe quedar expuesto a un interleaving dañino.**

**Visibilidad**

**Cuando los hilos se coordinan, necesitamos garantías sobre qué cambios de un hilo pueden observar los demás.**

**Nivel de esta presentación:**

✓ comprender estas dos necesidades
✓ utilizar después correctamente el mismo monitor

✗ Java Memory Model formal
✗ explicación física mediante cachés/RAM
✗ volatile
✗ APIs atómicas

**Todavía no hemos elegido la herramienta: primero sabemos qué propiedad necesitamos.**

# ¿Qué parte exacta necesita protección?

```
fun ingresar(cantidad: Int) {
    saldo += cantidad
}
```

## Sección crítica

Parte del programa que accede a estado compartido y cuya ejecución concurrente no controlada puede romper la corrección.

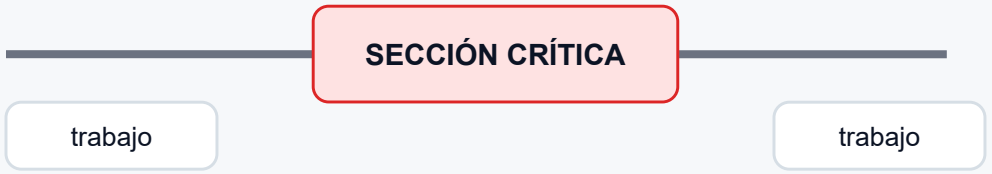
**saldo += cantidad**

**← sección crítica**

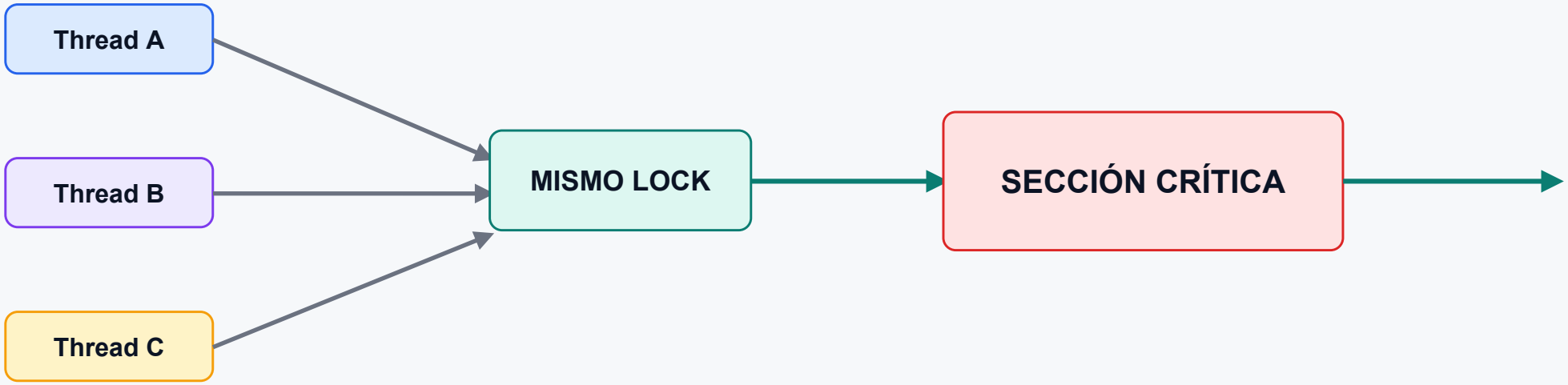
**LEER → CALCULAR → ESCRIBIR**

**No necesitamos convertir todo run() en sección crítica.**

**La operación crítica completa debe quedar protegida.**



# Exclusión mutua: uno dentro, los demás esperan




## Exclusión mutua

Respecto del mismo mecanismo de bloqueo, como máximo un hilo ejecuta la sección protegida cada vez.

A entra
B intenta entrar → espera
A sale
B puede entrar

**exclusión mutua ≠ eliminar toda la concurrencia**

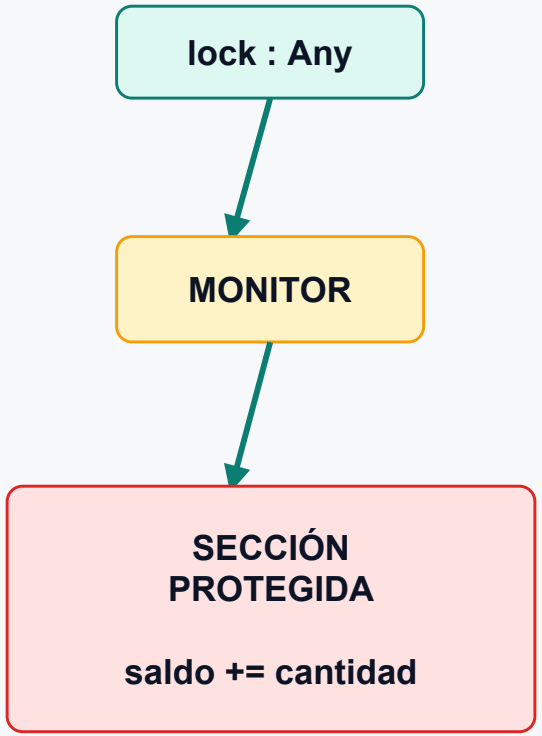
**Solo restringimos el acceso que necesita protección.**
**Objetivo: corrección.**

# synchronized(lock): proteger con un monitor JVM

```
class Cuenta {
    var saldo = 0
        private set

    private val lock = Any()

    fun ingresar(cantidad: Int) {
        synchronized(lock) {
            saldo += cantidad
        }
    }
}
```




```
private val lock = Any()
```

**Objeto estable cuyo monitor utilizamos para coordinar esa Cuenta.**

```
synchronized(lock) {
    ...
}
```

**El hilo debe adquirir el monitor asociado a lock antes de ejecutar el bloque.**

**mismo objeto lock → mismo monitor**

# La protección solo funciona si competimos por el mismo lock

## ✓ Correcto

```
class Cuenta {
    private val lock = Any()

    fun ingresar(cantidad: Int) {
        synchronized(lock) {
            saldo += cantidad
        }
    }
}
```

**Todas las llamadas a ingresar sobre esa misma Cuenta utilizan el mismo objeto lock.**

## ✗ Incorrecto

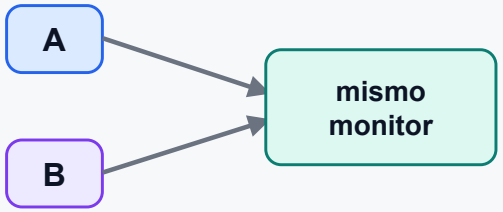
```
fun ingresar(cantidad: Int) {
    val lock = Any()

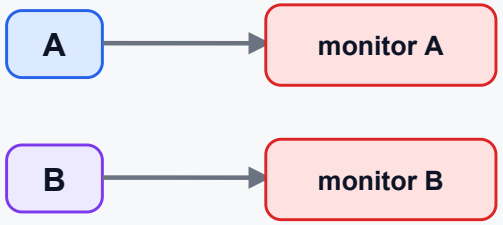
    synchronized(lock) {
        saldo += cantidad
    }
}
```

**Cada llamada crea un Any() nuevo.**

**llamada A → monitor A**
**llamada B → monitor B**

**Pueden entrar a la vez.**




**Regla clave**
**La exclusión mutua entre esos accesos existe si compiten por el mismo monitor.**

LEER → CALCULAR → ESCRIBIR
└──── actualización completa ────┘

# Repetimos: el orden puede variar, el saldo no

## La modificación esencial

```
private val lock = Any()

fun ingresar(cantidad: Int) {
    synchronized(lock) {
        saldo += cantidad
    }
}
```

start mete-10
start mete-5

---

join mete-10
join mete-5

---

comprobar saldo

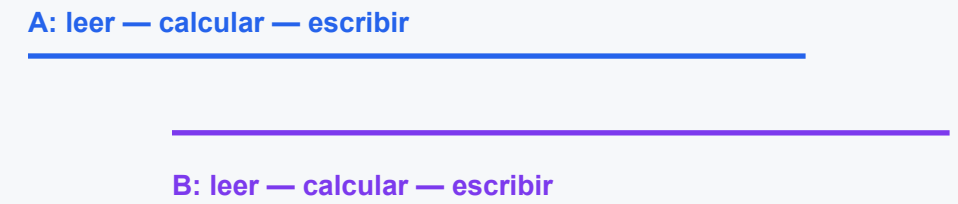
**Debe preservarse**

**saldo final = 15 000 000**

**No garantizamos**

**A → B → A → B → ...**

## Sin protección




## Con el mismo monitor



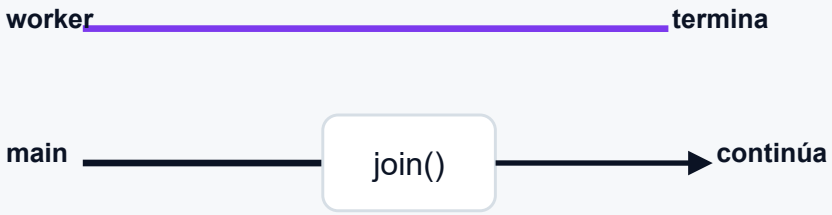

**Si todas las actualizaciones del saldo siguen este protocolo con el mismo lock, no pueden intercalarse dentro de la operación crítica.**

**Conexión con la presentación anterior:**
**A posee el monitor; B intenta adquirirlo → B puede estar en Thread.State.BLOCKED**

# join() y synchronized resuelven problemas diferentes

## join()

**¿Cuándo puede continuar el hilo que espera?**




- espera la terminación del Thread
- al retornar, permite observar las acciones completadas por ese hilo
- no excluye a A y B entre sí mientras trabajan

**synchronized → actualizaciones correctas**
**join → ambos terminados antes de comprobar**

## synchronized(lock)

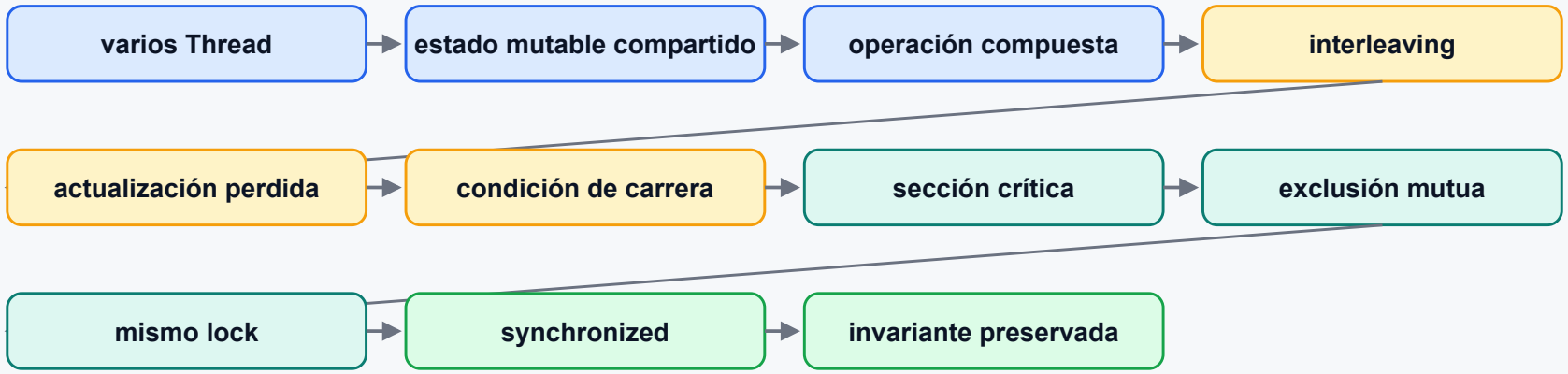
**¿Quién puede entrar ahora en la sección protegida?**




**Controla el acceso concurrente mediante el monitor.**

**join() no proporciona la exclusión mutua necesaria entre las actualizaciones concurrentes.**

# De «varios hilos» a «varios hilos correctos»




**join()**
↓
**esperar finalización antes de comprobar**

## Cinco preguntas para revisar código multihilo

1. ¿Qué estado comparten?
2. ¿Qué accesos pueden interferirse?
3. ¿Cuál es la sección crítica?
4. ¿Todos esos accesos siguen el mismo protocolo de protección?
5. ¿Cómo compruebo y documento la corrección?

**Una ejecución correcta no basta.**
**Debemos poder explicar por qué el programa es correcto.**

**En la próxima presentación**

**quién ejecuta · qué estado se comparte · cómo mantenemos la corrección**